Generating 6-D Trajectories for Omnidirectional Multirotor Aerial Vehicles in Cluttered Environments

Peiyan Liu, Yuanzhe Shen, Yueqian Liu, Fengyu Quan, Can Wang, *Senior Member, IEEE*, and Haoyao Chen, *Senior Member, IEEE*

***Abstract*—As fully-actuated systems, omnidirectional multirotor aerial vehicles (OMAVs) have more flexible maneuverability and advantages in aggressive flight in cluttered environments than traditional underactuated MAVs. This letter aims to achieve safe flight of OMAVs in cluttered environments. We present a 3-stage pipeline to generate safe and dynamically feasible 6-D rigid body trajectories for OMAVs. Given the obstacle information and goal points, we first search a feasible initial path and represent the 3-D collision-free regions around the path by a series of intersecting convex polyhedra. Then a general 6-D trajectory optimization framework considering safety and kinodynamic constraints is presented to finally generate a feasible 6-D $SE(3)$ trajectory, where the $SE(3)$ trajectory is represented as a 6-D polynomial in time without any inherent manifold constraints using rotation parameterization. A stereographic-based rotation parameterization is adopted to achieve more efficient optimization. To verify the proposed framework's performance, simulations and real-world experiments based on a tilt-rotor hexarotor aerial vehicle are carried out.**

***Index Terms*—Aerial systems: perception and autonomy, collision avoidance, motion and path planning.**

I. Introduction

MULTIROTOR aerial vehicles (MAVs) have stood out from various intelligent robots and entered our lives from laboratories. However, most traditional MAVs are underactuated systems, which means their translation and rotation dynamics are coupled. This nature limits the maneuverability of traditional MAVs. In order to fully exploit the potential of MAVs, several kinds of omnidirectional MAVs (OMAVs) with decoupled position and attitude control have been developed in recent years. By changing the configuration of rotors [1] or adding tilting degrees of freedom to rotors [2], this kind of MAV can perform controlled and free rigid body motion, which is impossible for traditional underactuated ones. In some extreme scenarios, such as a narrow straight passage, traditional MAVs coupling acceleration with attitude will be most likely unable to pass through it without collision, while OMAVs can tilt themselves to adapt to the narrow space by controlling the attitude and simultaneously, control its position to achieve smooth and collision-free passing. Such advantages make OMAVs bound to play a great application value in scenarios like aerial manipulation and disaster rescue.

In order to better exploit the potential of OMAVs, a proper trajectory generation framework is indispensable. It needs to generate 6-D $SE(3)$ trajectories and take into account the vehicle's shape and pose to adapt to cluttered environment. Moreover, the lightweight requirements of MAVs limit the available onboard computing power. So our method has higher requirements on computational load and efficiency than most classical motion planning methods applied in other robotic scenarios.

The existing research on OMAVs mainly focuses on the mechanical structure design and control strategy, but few achievements have been made in trajectory generation. Brescianini et al. efficiently generate 6-D trajectories that satisfy certain input constraints for OMAVs using motion primitives. An energy-efficient trajectory generation method for a tilt-rotor hexarotor UAV is proposed in [3]. Pantic et al. [4] present a trajectory generation method based on Riemannian Motion Policies (RMPs), which aims to drive a vehicle to fly to and along a specified surface and is applied to aerial physical interaction. The above works do not take into account the obstacles in the environment.

The existing works on trajectory generation of OMAVs do not meet our requirements well, while 3-D position trajectory generation are relatively mature for traditional underactuated MAVs. Mellinger et al. [5] generate smooth trajectories by minimizing the square integral of the trajectory derivatives for the first time. Several efficient schemes have been created based on the idea in [5]. In order to meet the safety requirements, gradient information in the environment is used in [6] and [7] to push the trajectories away from obstacles to achieve collision avoidance. Another common way is to use intersecting geometry primitives to approximate the collision-free regions [8], [9], [10], [11], and the union of these primitives is called a safe flight corridor (SFC). Wang et al. [12] propose an optimization-based trajectory generation framework for multicopters. It shows state-of-the-art performance in efficiency, extensibility, and solution quality. Based on [12], several whole-body $SE(3)$ trajectory generation methods are proposed [10], [11], [13] in which whole-body safety constraints can be constructed and handled efficiently

Received 10 April 2024; accepted 2 August 2024. Date of publication 22 August 2024; date of current version 9 September 2024. This article was recommended for publication by Associate Editor M. W. Mueller and Editor G. Loianno upon evaluation of the reviewers' comments. This work was supported in part by the National Natural Science Foundation of China under Grant U21A20119 and Grant U1713206 and in part by Shenzhen Science and Innovation Committee under Grant RCJC20231211090050082, Grant SZXJP20230703093206015, and Grant JCYJ20210324120400003. *(Corresponding authors: Can Wang; Haoyao Chen.)*

Peiyan Liu, Yuanzhe Shen, Fengyu Quan, and Haoyao Chen are with the School of Mechanical Engineering and Automation, Harbin Institute of Technology Shenzhen, Harbin 150001, China (e-mail: hychen5@hit.edu.cn).

Yueqian Liu is with the Faculty of Aerospace Engineering, Delft University of Technology, 2628 CD Delft, The Netherlands (e-mail: y.liu-133@student.tudelft.nl).

Can Wang is with the Shenzhen Institute of Advanced Technology, Chinese Academy of Sciences, Shenzhen 518055, China (e-mail: can.wang@siat.ac.cn).

This letter has supplementary downloadable material available at https://doi.org/10.1109/LRA.2024.3448135, provided by the authors.

Digital Object Identifier 10.1109/LRA.2024.3448135

2377-3766 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 06:31:03 UTC from IEEE Xplore. Restrictions apply.

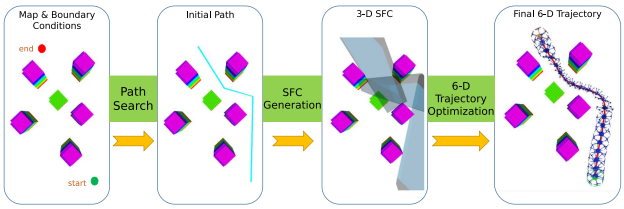


Fig. 1. The proposed 3-stage 6-D trajectory generation framework involves initial path search, SFC generation, and 6-D trajectory optimization.

with the polyhedral SFC. However, these methods are only suitable for under-actuated MAVs whose position and attitude are highly coupled.

This letter proposes an efficient 3-stage 6-D trajectory generation framework for OMAVs in cluttered environments, as shown in Fig. 1. Similar to [11], 3-D polyhedral SFC is used to represent the collision-free region. First, we use sampling-based methods like RRT [14] to search for an initial feasible path from the start to the end point and generate a 3-D SFC based on the initial path. Then, the 6-D trajectory optimization stage efficiently generates a smooth, safe, and dynamically feasible 6-D trajectory connecting the start and the end states with linear time and space complexity. For trajectory representation, we represent a vehicle's attitude as a Hamilton unit quaternion $\mathbf{Q} \in \mathbb{S}^3$ [15] and parameterize it into a free 3-D vector using stereographic projection to eliminate the constraints inherent in the $\mathbb{S}^3$ manifold. This 3-D attitude vector is combined with the 3-D position vector to form a 6-D pose vector. The complete $SE(3)$ trajectory is then represented as a piece-wise polynomial of the pose vector over time. The vehicle's shape and attitude are taken into account to apply whole-body safety constraints to the trajectory. We formulate the trajectory generation problem as a constrained optimization problem and finally transform it into an unconstrained one that can be solved using quasi-Newton methods. In summary, the main contributions of this letter are:

- Based on 3-D rotation parameterization and the idea of the state-of-the-art trajectory optimization method [12] for traditional underactuated MAVs, we present a general 6-D rigid body trajectory optimization framework considering whole-body safety and dynamic limits for OMAVs where rotation trajectories are represented as curves in $\mathbb{R}^3$ through rotation parameterization.
- We achieve efficient trajectory optimization by adopting a stereographic-projection-based rotation parameterization.
- By integrating the proposed trajectory optimization method with path search and SFC generation, we propose an efficient 6-D trajectory generation pipeline (Fig. 1) that leverages the full potential of OMAVs for obstacle avoidance for the first time.

II. System Modeling and Control

A. Definitions

The methodology of this letter mainly involves two right-handed coordinate systems: the world (inertial) frame $\mathcal{F}_W$ and the body frame $\mathcal{F}_b$ (Fig. 2). We denote $\mathbf{a}_A$ the coordinate of a vector $\mathbf{a}$ expressed in frame $\mathcal{F}_A$, and we omit the subscript if $\mathcal{F}_A$ is the world frame $\mathcal{F}_W$. Denote $\mathbf{R}$ the rotation matrix of the body frame $\mathcal{F}_b$ w.r.t. the world frame $\mathcal{F}_W$, and thus $\mathbf{a} = \mathbf{R}\mathbf{a}_b$.

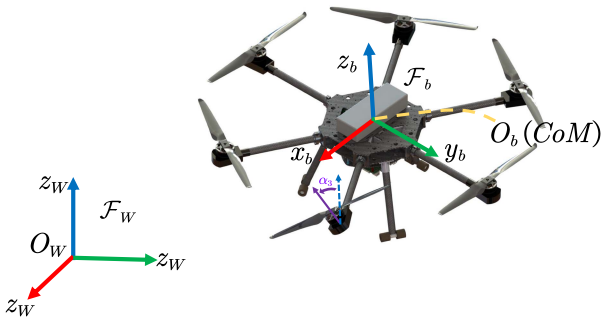


Fig. 2. This figure shows the frame definitions, the rotor tilt angle, and the CAD model of OmniHex. $\mathcal{F}_W$ uses the east-north-up (ENU) coordinate system. The origin of $\mathcal{F}_b$ coincides with the vehicle's Center of Mass (CoM); the x_b axis points forward, the y_b axis points to the body's left, and the right-hand rule determines the z_b axis.

B. System Modeling of OMAVs

A common OMAV has six independent control degrees of freedom, we take joint thrust and torque generated by the rotors expressed in $\mathcal{F}_b$ as its control input

$$\mathbf{u} := \begin{bmatrix} \mathbf{f}_b^\top & \boldsymbol{\tau}_b^\top \end{bmatrix}^\top \in \mathbb{R}^6, \tag{1}$$

and we select CoM position $\mathbf{p}$, CoM velocity $\mathbf{v} = \dot{\mathbf{p}}$, $\mathcal{F}_b$'s orientation $\mathbf{Q}$ (expressed as Hamilton unit quaternion [15]), and $\mathcal{F}_b$'s angular velocity $\boldsymbol{\omega}$ (all expressed in $\mathcal{F}_W$) as its state variables:

$$\mathbf{x} := [\underbrace{p_x, p_y, p_z}_{\mathbf{p}^\top}, \underbrace{Q_w, Q_x, Q_y, Q_z}_{\mathbf{Q}^\top}, \underbrace{v_x, v_y, v_z}_{\mathbf{v}^\top}, \underbrace{\omega_x, \omega_y, \omega_z}_{\boldsymbol{\omega}^\top}]^\top. \tag{2}$$

where $\mathbf{x} \in \mathbb{R}^{13}$ is the full state vector. The system's output is given by the position of its CoM and the orientation of $\mathcal{F}_b$ expressed in $\mathcal{F}_W$:

$$\mathbf{y}(t) := \{\mathbf{p}(t), \mathbf{Q}(t)\} \in SE(3). \tag{3}$$

Expressing a rotation trajectory as a function of a unit quaternion or a rotation matrix w.r.t. time t is intuitive. However, this implies equality constraints inherent in $SO(3)$, which can be troublesome for trajectory optimization. So, we consider parameterizing each rotation $\mathbf{Q}$ as an unconstrained 3-D vector. Denote a rotation trajectory by a curve $\boldsymbol{\sigma}(t) : [t_0, t_M] \to \mathbb{R}^3$, and the corresponding $\mathbf{Q}(t)$ is determined by a smooth surjection $\mathbf{Q}(\boldsymbol{\sigma}) : \mathbb{R}^3 \to SO(3)$. Then, we express the 6-D trajectory as

$$\mathbf{z}(t) := \begin{bmatrix} \mathbf{p}^\top(t) & \boldsymbol{\sigma}^\top(t) \end{bmatrix}^\top \in \mathbb{R}^6. \tag{4}$$

The angular velocity $\boldsymbol{\omega}$ of $\mathcal{F}_b$ can be calculated as

$$\boldsymbol{\omega}^\wedge = \boldsymbol{\omega}^\wedge(\boldsymbol{\sigma}, \dot{\boldsymbol{\sigma}}) = \sum_{i=1}^{3} \frac{\partial \mathbf{R}(\boldsymbol{\sigma})}{\partial \sigma_i} \dot{\sigma}_i \mathbf{R}(\boldsymbol{\sigma})^\top, \tag{5}$$

where $\mathbf{R}$ is the rotation matrix determined by $\mathbf{Q}$; $(\cdot)^\wedge : \mathbb{R}^3 \to \mathfrak{so}(3)$ denotes the skew-symmetric matrix of a 3-D vector and thus $\mathbf{a} \times \mathbf{b} = \mathbf{a}^\wedge \mathbf{b}, \forall \mathbf{a}, \mathbf{b} \in \mathbb{R}^3$. The inverse map is denoted as $(\cdot)^\vee : \mathfrak{so}(3) \to \mathbb{R}^3$. The angular acceleration $\dot{\boldsymbol{\omega}}$ can be further

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 06:31:03 UTC from IEEE Xplore. Restrictions apply.

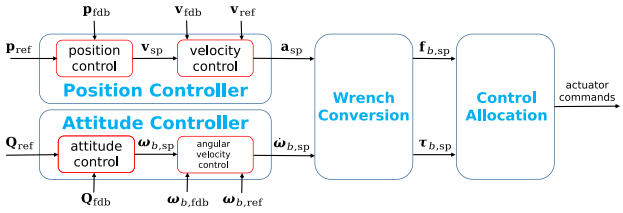


Fig. 3. The control pipeline of OmniHex. As the position control and attitude control of OmniHex is decoupled, the linear and angular acceleration setpoints are calculated separately by their respective cascade PID controllers.

calculated according to (5) :

$$(\dot{\boldsymbol{\omega}})^{\wedge} = \sum_{i=1}^{3}\left(\sum_{j=1}^{3}\frac{\partial^2\mathbf{R}}{\partial\sigma_i\partial\sigma_j}\ddot{\sigma}_j\dot{\sigma}_i + \frac{\partial\mathbf{R}}{\partial\sigma_i}\ddot{\sigma}_i\right)\mathbf{R}^{\top} + \dot{\mathbf{R}}\dot{\mathbf{R}}^{\top}. \quad (6)$$

Thus, we obtain the expression of the state $\mathbf{x}$ w.r.t. $\mathbf{z}$ and $\mathbf{z}$'s finite derivatives, given as

$$\mathbf{x} = \begin{bmatrix}\mathbf{p}\\ \mathbf{Q}\\ \mathbf{v}\\ \boldsymbol{\omega}\end{bmatrix} = \Psi_x(\mathbf{z},\dot{\mathbf{z}}) := \begin{bmatrix}\mathbf{p}\\ \mathbf{Q}(\boldsymbol{\sigma})\\ \dot{\mathbf{p}}\\ \left(\sum_{i=1}^{3}\frac{\partial\mathbf{R}}{\partial\sigma_i}\dot{\sigma}_i\mathbf{R}^{\top}\right)^{\vee}\end{bmatrix}. \quad (7)$$

The expression of the control input $\mathbf{u}$ w.r.t. $\mathbf{z}$ and $\mathbf{z}$'s finite derivatives can be determined by combining (5) and (6) with Newton-Euler equation, given as

$$\mathbf{u} = \begin{bmatrix}\mathbf{f}_b\\ \boldsymbol{\tau}_b\end{bmatrix} = \Psi_u(\mathbf{z},\dot{\mathbf{z}},\ddot{\mathbf{z}}) := \begin{bmatrix}m\mathbf{R}(\boldsymbol{\sigma})^{\top}(\ddot{\mathbf{p}}-\mathbf{g})\\ \mathbf{R}(\boldsymbol{\sigma})^{\top}\left(\boldsymbol{\omega}^{\wedge}\mathbf{J}(\boldsymbol{\sigma})\boldsymbol{\omega} + \mathbf{J}(\boldsymbol{\sigma})\dot{\boldsymbol{\omega}}\right)\end{bmatrix}, \quad (8)$$

where m is the mass of the system; $\mathbf{g} = \begin{bmatrix}0 & 0 & -9.8\end{bmatrix}^{\top}\ \text{m}\cdot\text{s}^{-2}$ is acceleration of gravity in $\mathcal{F}_W$; $\mathbf{J}(\boldsymbol{\sigma}) := \mathbf{R}(\boldsymbol{\sigma})\mathbf{J}_b\mathbf{R}(\boldsymbol{\sigma})^{\top} \in \mathbb{R}^{3\times3}$, where $\mathbf{J}_b \in \mathbb{R}^{3\times3}$ is the vehicle's inertia matrix in $\mathcal{F}_b$ which can be treated as a constant.

Equations (7) and (8) indicate that our selected trajectory representation $\mathbf{z}(t)$ has the properties as a flat output [16]. Therefore, it is convenient to transform constraints on the state $\mathbf{x} \in \mathbb{R}^{13}$ and the input $\mathbf{u} \in \mathbb{R}^6$ to constraints on $\mathbf{z} \in \mathbb{R}^6$ and $\mathbf{z}$'s finite derivatives using (7) and (8).

C. Position and Attitude Control

In this section, we briefly introduce the control strategy designed for a tilt-rotor omnidirectional hexacopter (from now on called OmniHex) to accurately track 6-D trajectories. As shown in Fig. 2, compared to traditional hexacopters, OmniHex adds an additional controllable degree of freedom for each rotor to rotate around the axis of the arm on which it is mounted, which allows OmniHex to generate thrust and torque in any direction relative to $\mathcal{F}_b$ for independent control of position and attitude. The control pipeline of OmniHex is shown in Fig. 3. In the following statements, the quantities with subscript "ref" are the setpoints given by the reference trajectory, and those with subscript "fdb" are the feedback from the state estimator.

1) Position Controller: The outer loop of the position controller is a proportional controller that sets the desired velocity $\mathbf{v}_{\text{sp}}$ for the inner loop based on position error $\boldsymbol{e}_p := \mathbf{p}_{\text{ref}} - \mathbf{p}_{\text{fdb}}$.

$$\mathbf{v}_{\text{sp}} = \mathbf{K}_{P,p}\boldsymbol{e}_p, \quad (9)$$

$\mathbf{K}_{P,p} \in \mathbb{R}^{3\times3}$ is the diagonal gain matrix. The inner velocity control loop uses PID control with a feed-forward design, given as

$$\mathbf{a}_{\text{sp}} = \mathbf{K}_{P,v}\boldsymbol{e}_v + \mathbf{K}_{I,v}\int\boldsymbol{e}_v\mathrm{d}t + \mathbf{K}_{D,v}\frac{\mathrm{d}\boldsymbol{e}_v}{\mathrm{d}t} + \mathbf{K}_{F,v}\mathbf{v}_{\text{ref}}, \quad (10)$$

where $\mathbf{K}_{P,v}, \mathbf{K}_{I,v}, \mathbf{K}_{D,v}, \mathbf{K}_{F,v} \in \mathbb{R}^{3\times3}$ are diagonal gain matrices for proportional, integral, derivative, and feed-forward control, respectively. $\boldsymbol{e}_v := \boldsymbol{v}_{\text{sp}} - \boldsymbol{v}_{\text{fdb}}$ is the velocity error.

2) Attitude Control: We use Hamilton unit quaternions to represent the attitudes. The attitude error is defined as the difference between the reference attitude $\mathbf{Q}_{\text{ref}}$ and the attitude feedback $\mathbf{Q}_{\text{fdb}}$

$$\boldsymbol{e}_Q := \begin{bmatrix}e_{Q,w} & \boldsymbol{e}_{Q,\text{vec}}^{\top}\end{bmatrix}^{\top} = \mathbf{Q}_{\text{ref}} \otimes \mathbf{Q}_{\text{fdb}}^{-1}, \quad (11)$$

where $e_{Q,w} \in \mathbb{R}$ and $\boldsymbol{e}_{Q,\text{vec}} \in \mathbb{R}^3$ are scalar and vector parts of quaternion $\boldsymbol{e}_Q$, respectively. $\otimes$ denotes quaternion product. Then the outer loop of attitude control maps the attitude error to the desired angular velocity $\boldsymbol{\omega}_{b,\text{sp}}$ (expressed in $\mathcal{F}_b$) as follows:

$$\boldsymbol{\omega}_{b,\text{sp}} = \text{sign}(e_{Q,w})\cdot\mathbf{K}_{P,Q}\boldsymbol{e}_{Q,\text{vec}}, \quad (12)$$

Denoting angular velocity error as $\boldsymbol{e}_\omega := \boldsymbol{\omega}_{b,\text{ref}} - \boldsymbol{\omega}_{b,\text{fdb}}$, the inner loop calculates the desired angular acceleration as follows:

$$\dot{\boldsymbol{\omega}}_{b,\text{sp}} = \mathbf{K}_{P,\omega}\boldsymbol{e}_\omega + \mathbf{K}_{I,\omega}\int\boldsymbol{e}_\omega\mathrm{d}t + \mathbf{K}_{D,\omega}\frac{\mathrm{d}\boldsymbol{e}_\omega}{\mathrm{d}t} + \mathbf{K}_{F,\omega}\boldsymbol{\omega}_{b,\text{ref}}. \quad (13)$$

3) Wrench Conversion and Control Allocation: Obtaining the desired accelerations, we can calculate the desired control wrench $\mathbf{w}_{b,\text{sp}}$ (expressed in $\mathcal{F}_b$) as follows:

$$\mathbf{w}_{b,\text{sp}} := \begin{bmatrix}\mathbf{f}_{b,\text{sp}}\\ \boldsymbol{\tau}_{b,\text{sp}}\end{bmatrix} = \begin{bmatrix}m\mathbf{Q}_{\text{fdb}}^{-1}\otimes(\mathbf{a}_{\text{sp}}-\mathbf{g})\otimes\mathbf{Q}_{\text{fdb}}\\ \mathbf{J}_b\dot{\boldsymbol{\omega}}_{b,\text{sp}} + \mathbf{d}_{\text{com}}^{\wedge}\mathbf{f}_{b,\text{sp}} + \boldsymbol{\omega}_{b,\text{fdb}}^{\wedge}\mathbf{J}_b\boldsymbol{\omega}_{b,\text{fdb}}\end{bmatrix}, \quad (14)$$

where the offset of the CoM $\mathbf{d}_{\text{com}}$ is predefined. The actuator commands of OmniHex are rotation speeds ω_i and tilt angles $\alpha_i, i = 1,\ldots,6$ of its six rotors. Defining $\cos\alpha_i = c_i, \sin\alpha_i = s_i, i = 1,\ldots,6$, $\mathbf{w}_{b,\text{sp}}$ calculated in (14) can also be expressed as

$$\mathbf{w}_{b,\text{sp}} = \boldsymbol{A}[\omega_1^2 s_1, \omega_1^2 c_1, \ldots, \omega_6^2 s_6, \omega_6^2 c_6]^{\top}, \quad (15)$$

where $\boldsymbol{A} \in \mathbb{R}^{6\times12}$ is the allocation matrix defined in [17]. Using the strategy presented in [17], we can obtain the desired actuator commands.

III. Method

This section presents the details of the proposed 3-stage optimization-based trajectory generation framework based on the above system modeling. The path search and the SFC generation stages are introduced briefly in Section III-A. The trajectory optimization stage is described in detail in Sections III-B to III-C, which is the main contribution of this letter.

A. Whole-Body Safety Constraint

To construct whole-body safety constraints, we use convex polyhedral SFC to represent the collision-free regions in 3-D position space connecting the start point $\mathbf{p}_o \in \mathbb{R}^3$ and the target point $\mathbf{p}_f \in \mathbb{R}^3$. First, we search an initial collision-free path

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 06:31:03 UTC from IEEE Xplore. Restrictions apply.

$\mathbf{p}_0 \to \mathbf{p}_1 \to \cdots \to \mathbf{p}_{M_\mathcal{P}}$ from $\mathbf{p}_0 = \mathbf{p}_o$ to $\mathbf{p}_{M_\mathcal{P}} = \mathbf{p}_f$. Then, we use RILS [18] to find a 3-D convex polyhedron $\mathcal{P}_i$ approximating the collision-free region around each path segment $\mathbf{p}_{i-1} \to \mathbf{p}_i, i = 1, \ldots, M_\mathcal{P}$ as a primitive of SFC. The resulting SFC is defined as $\mathcal{SFC} := \bigcup_{i=1}^{M_\mathcal{P}} \mathcal{P}_i$. Two adjacent convex polyhedral primitives satisfy the connection condition:

$$(\mathcal{P}_i \cap \mathcal{P}_{i+1})^\circ \neq \emptyset, i = 1, \ldots, M_\mathcal{P} - 1, \tag{16}$$

where $(A)^\circ$ denotes the interior of the point set A.

Similar to [11], the shape of the vehicle is approximated by a convex polyhedron $\mathcal{P}_\mathrm{S}$ that is fixed to $\mathcal{F}_b$ and wraps the entire vehicle. The coordinates of its vertices in $\mathcal{F}_b$ denoted by $\tilde{\boldsymbol{v}}_l$, $l = 1, \cdots L_\mathrm{v}$ are known constants. The safety of the vehicle can be guaranteed as long as the L_v vertices are all in the convex polyhedron $\mathcal{P}$ that represents the safe region. We express $\mathcal{P}$ using linear inequalities:

$$\mathcal{P} := \{\mathbf{p} \in \mathbb{R}^3 | \mathbf{n}_k^\top \mathbf{p} - d_k \leq 0, k = 1, \ldots, K\}, \tag{17}$$

which means that the convex polyhedron $\mathcal{P}$ is the intersection of K halfspaces. $\mathbf{n}_k \in \mathbb{R}^3$ is the unit outer normal vector of the k-th halfspace. Then the safety condition of the vehicle with position $\mathbf{p}$ and attitude $\mathbf{R}$ can be written as follow

$$\mathbf{n}_k^\top \boldsymbol{v}_l - d_k \leq 0, \forall k \in \{1, \ldots, K\}, \forall l \in \{1, \ldots, L_\mathrm{v}\}, \tag{18}$$

where $\boldsymbol{v}_l := \mathbf{p} + \mathbf{R}\tilde{\boldsymbol{v}}_l, l = 1, \cdots L_\mathrm{v}$ are coordinates of vertices of $\mathcal{P}_\mathrm{S}$ in $\mathcal{F}_W$.

B. 6-D Trajectory Optimization

For the convenience of controlling the derivatives, we express $\mathbf{z}(t)$ as a 6-D piece-wise polynomial over time:

$$\mathbf{z}(t) = \mathbf{c}_i^\top \boldsymbol{\beta}(t - t_{i-1}), \text{if } t \in [t_{i-1}, t_i], i = 1, \ldots, M \tag{19}$$

where $\mathbf{c}_i \in \mathbb{R}^{(n+1)\times 6}$ is the coefficient matrix of the i-th piece and $\boldsymbol{\beta}(\alpha) := \begin{bmatrix} 1 & \alpha & \cdots & \alpha^n \end{bmatrix}^\top \in \mathbb{R}^{(n+1)}$. M is the number of pieces. The coefficient matrix of the whole trajectory $\mathbf{c}$ and the times allocated for all the pieces are defined as

$$\mathbf{c} := \begin{bmatrix} \mathbf{c}_1^\top & \cdots & \mathbf{c}_M^\top \end{bmatrix}^\top \in \mathbb{R}^{M(n+1)\times 6}, \tag{20}$$

$$\begin{aligned} \mathbf{T} &:= \begin{bmatrix} t_1 - t_0 & \cdots & t_M - t_{M-1} \end{bmatrix}^\top \\ &= \begin{bmatrix} T_1 & \cdots & T_M \end{bmatrix}^\top \in \mathbb{R}_+^M. \end{aligned} \tag{21}$$

Our goal is to find the optimal $\mathbf{c}$ and $\mathbf{T}$ that minimize a given objective function. We adopt MINCO trajectory representation proposed in [12] to deal with the high dimensionality of $\mathbf{c}$ which may reduce optimization efficiency. By enforcing the optimality conditions in Theorem 2 of [12], the parameters of $\mathbf{z}(t)$ can be transformed from $(\mathbf{c}, \mathbf{T})$ to the waypoints $\mathbf{q} := \begin{bmatrix} \mathbf{q}_1 & \cdots & \mathbf{q}_{M-1} \end{bmatrix} \in \mathbb{R}^{6\times(M-1)}, i = 1, \ldots, M-1$ and $\mathbf{T}$. After specifying $(\mathbf{q}, \mathbf{T})$, the start condition $\bar{\mathbf{z}}_o := \mathbf{z}^{[s-1]}(t_0) := \begin{bmatrix} \mathbf{z}(t_0)^\top & \cdots & \mathbf{z}^{(s-1)}(t_0)^\top \end{bmatrix}^\top \in \mathbb{R}^{6s}$, and the end condition $\bar{\mathbf{z}}_f := \mathbf{z}^{[s-1]}(t_0) := \begin{bmatrix} \mathbf{z}(t_M)^\top & \cdots & \mathbf{z}^{(s-1)}(t_M)^\top \end{bmatrix}^\top \in \mathbb{R}^{6s}$, $\mathbf{c}$ will be determined uniquely in an efficient way by solving the sparse linear system

$$\mathbf{M}(\mathbf{T})\mathbf{c}(\mathbf{q}, \mathbf{T}) = \mathbf{b}(\mathbf{q}) \Rightarrow \mathbf{c}(\mathbf{q}, \mathbf{T}) = \mathbf{M}^{-1}(\mathbf{T})\mathbf{b}(\mathbf{q}), \tag{22}$$

where $\mathbf{M} \in \mathbb{R}^{2Ms\times 2Ms}$ and $\mathbf{b} \in \mathbb{R}^{2Ms\times 6}$ are defined in [12]. The resulting trajectory satisfies $\mathbf{z}(t_i) = \mathbf{q}_i$ and is $2s-2$ times continuously differentiable at $t_i, i = 1, \ldots, M-1$. The degree of polynomial $\mathbf{z}(t)$ is determined by the system order $s \in \mathbb{N}_+$ as $n = 2s - 1$.

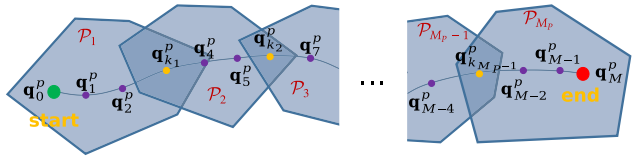


Fig. 4. The 2-D illustration of spatial constraints (23d)–(23f). In this case $k_1 = 3, k_2 = 6$, and $k_{M_{\mathcal{P}-1}} = M - 3$.

For the convenience of the following statement, we divide $\mathbf{c}$ and $\mathbf{q}$ into position blocks and attitude blocks: $\mathbf{c}_i = \begin{bmatrix} \mathbf{c}_i^p & \mathbf{c}_i^\sigma \end{bmatrix}, \mathbf{c}_i^p, \mathbf{c}_i^\sigma \in \mathbb{R}^{2s\times 3}$; $\mathbf{c} = \begin{bmatrix} \mathbf{c}^p & \mathbf{c}^\sigma \end{bmatrix}, \mathbf{c}^p, \mathbf{c}^\sigma \in \mathbb{R}^{2Ms\times 3}$; $\mathbf{q}_i = [\mathbf{q}_i^{p\top} \quad \mathbf{q}_i^{\sigma\top}]^\top, \mathbf{q}_i^p, \mathbf{q}_i^\sigma \in \mathbb{R}^3$; $\mathbf{q} = [\mathbf{q}^{p\top} \quad \mathbf{q}^{\sigma\top}]^\top, \mathbf{q}^p, \mathbf{q}^\sigma \in \mathbb{R}^{3\times(M-1)}$.

We expect the trajectory $\mathbf{z}(t)$ to be smooth enough and satisfy the boundary conditions, the dynamic constraints, and the safety constraints. Then, the original form of our trajectory optimization problem is as follows:

$$\min_{\mathbf{q},\mathbf{T}} \int_{t_0}^{t_M} \|\mathbf{z}^{(s)}(t)\|_2^2 \mathrm{d}t + k_\rho \|\mathbf{T}\|_1, \tag{23a}$$

$$s.t. \quad \mathbf{z}^{[s-1]}(t_0) = \bar{\mathbf{z}}_o, \mathbf{z}^{[s-1]}(t_M) = \bar{\mathbf{z}}_f, \tag{23b}$$

$$\mathbf{z}(t) = \mathbf{c}_i^\top(\mathbf{q}, \mathbf{T})\boldsymbol{\beta}(t - t_{i-1}), \forall t \in [t_{i-1}, t_i], \quad i = 1, \ldots, M, \tag{23c}$$

$$\mathbf{q}_{k_i}^p \in (\mathcal{P}_i \cap \mathcal{P}_{i+1})^\circ, i = 1, \ldots, M_\mathcal{P} - 1, \tag{23d}$$

$$\mathbf{q}_j^p \in \mathcal{P}_1, \text{if } 1 \leq j < k_1, \tag{23e}$$

$$\mathbf{q}_j^p \in \mathcal{P}_{M_\mathcal{P}}, \text{if } k_{M_\mathcal{P}-1} < j \leq M - 1, \tag{23f}$$

$$\mathbf{q}_j^p \in \mathcal{P}_i, \text{if } k_{i-1} < j < k_i, i = 2, \ldots, M_\mathcal{P} - 1, \tag{23g}$$

$$\mathbf{T} \succ \mathbf{0}, \tag{23h}$$

$$\|\dot{\mathbf{p}}(t)\|_2^2 \leq v_{\max}^2, \forall t \in [t_0, t_M], \tag{23i}$$

$$\|\ddot{\mathbf{p}}(t)\|_2^2 \leq a_{\max}^2, \forall t \in [t_0, t_M], \tag{23j}$$

$$\|\boldsymbol{\omega}(t)\|_2^2 \leq \omega_{\max}^2, \forall t \in [t_0, t_M], \tag{23k}$$

$$\boldsymbol{v}_l(t) \in \mathcal{P}^i, \forall t \in [t_{i-1}, t_i], i = 1, \cdots M; l = 1, \ldots, L_\mathrm{v}.$$

The first term of the objective function (23a) is the smoothness cost, from now on denoted as J, and the second is the time regularization term. Spatial constraints (23d)–(23f) bind the position waypoints to a specific region in the SFC, where $k_i, i = 1, \ldots, M_\mathcal{P}$ is the indices of waypoints that should be confined to $(\mathcal{P}_i \cap \mathcal{P}_{i+1})^\circ$ and $1 \leq k_i < k_{i+1} \leq M - 1, i = 1, \ldots, M_\mathcal{P} - 1$, as illustrated in Fig. 4. Temporal constraint (23g) ensures that the time allocated to each piece is strictly positive. Inequalities (23h)–(23j) are kinodynamic constraints according to the task requirements and vehicle limits; Safety constraint (23k) confines each trajectory piece to a polyhedron primitive of SFC. We let $\mathcal{P}^i := \{\mathbf{p} \in \mathbb{R}^3 | \mathbf{n}_{i,k}^\top \mathbf{p} - d_{i,k} \leq 0, k = 1, \ldots, K_i\}$ denote the polyhedron to which the i-th piece

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 06:31:03 UTC from IEEE Xplore. Restrictions apply.

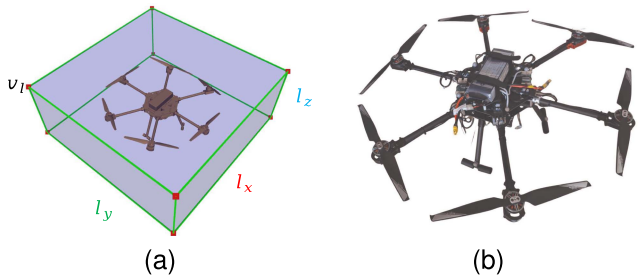


Fig. 5. Illustrations of some experiment settings. (a) Shows the cuboid used to approximate the vehicle's shape, of which the center coincides with the vehicle's CoM, and the three symmetry axes are parallel with axes of $\mathcal{F}_b$. (b) Shows the real OmniHex we develop.

is assigned. Note that in addition to (23h)–(23k), we can also add more continuous time constraints according to actual needs, such as dealing with the vehicle's actuator delays or dynamic changes in the environment.

The original trajectory optimization problem (23) contains various constraints. To deal with them, we draw on the ideas in [12]. The continuous-time constraints (23h) to (23k) can be softened as integral penalty terms. The spatial constraints (23d)–(23f) and temporal constraint (23g) can be eliminated using certain diffeomorphisms $\mathbf{q}^p(\boldsymbol{\xi})$ and $\mathbf{T}(\boldsymbol{\tau})$ suggested in [12]. Note that different from [12], our optimization variables include rotation waypoints $\mathbf{q}^\sigma$. Our strategy is not to impose any hard constraints on $\mathbf{q}^\sigma$, so there is no need to transform it. Finally, what we need to solve is an unconstrained optimization problem as follows:

$$\begin{aligned}\min_{\boldsymbol{\xi},\mathbf{q}^\sigma,\boldsymbol{\tau}}\ & J(\mathbf{q}^p(\boldsymbol{\xi}),\mathbf{q}^\sigma,\mathbf{T}(\boldsymbol{\tau}))+k_\rho\|\mathbf{T}(\boldsymbol{\tau})\|_1\\ &+\mathcal{W}_v\sum_{i=1}^{M}\sum_{j=1}^{\kappa}\mathcal{V}\left(\left\|\dot{\mathbf{p}}\left(\hat{t}_{ij}\right)\right\|_2^2-v_{\max}^2\right)\frac{T_i}{\kappa}\\ &+\mathcal{W}_a\sum_{i=1}^{M}\sum_{j=1}^{\kappa}\mathcal{V}\left(\left\|\ddot{\mathbf{p}}\left(\hat{t}_{ij}\right)\right\|_2^2-a_{\max}^2\right)\frac{T_i}{\kappa}\\ &+\mathcal{W}_\omega\sum_{i=1}^{M}\sum_{j=1}^{\kappa}\mathcal{V}\left(\left\|\boldsymbol{\omega}\left(\hat{t}_{ij}\right)\right\|_2^2-\omega_{\max}^2\right)\frac{T_i}{\kappa}\\ &+\mathcal{W}_c\sum_{i=1}^{M}\sum_{j=1}^{\kappa}\sum_{l=1}^{L_{\mathrm{v}}}\sum_{k=1}^{K_i}\mathcal{V}\left(\mathbf{n}_{i,k}^\top\boldsymbol{v}_l\left(\hat{t}_{ij}\right)-d_{i,k}\right)\frac{T_i}{\kappa},\end{aligned}\tag{24}$$

where $\mathcal{W}_\star$ is the weight of the corresponding penalty term. $\kappa\in\mathbb{N}_+$ controls the resolution of numerical integration. $\mathcal{V}(\cdot):=\max(\cdot,0)^3$ measures the constraint violation on the trajectory at the sampling time $\hat{t}_{ij}:=t_{i-1}+\frac{j}{\kappa}T_i$. Calculating the cost function and its gradients is $O(M)$ complex in time and space, significantly reducing the computational burden. With closed-form gradients, problem (24) can be efficiently solved by quasi-Newton methods.

C. Rotation Parameterization

From Section III-B we can find that choosing an appropriate rotation parameterization map $\mathbf{R}(\boldsymbol{\sigma})$ is essential for our trajectory generation. There are several commonly used ways to parameterize a rotation in $SO(3)$ as a vector in $\mathbb{R}^3$, such as Euler angles and axis-angle (also known as a Lie algebra of $SO(3)$). However, since $\boldsymbol{\sigma}$ is a free vector on which we do not apply any hard constraints, it will be ambiguous if $\boldsymbol{\sigma}$ represents Euler angles or an axis-angle: two very different $\boldsymbol{\sigma}$ values will most likely correspond to the same rotation. Moreover, Euler angle representation has the problem of gimbal lock.

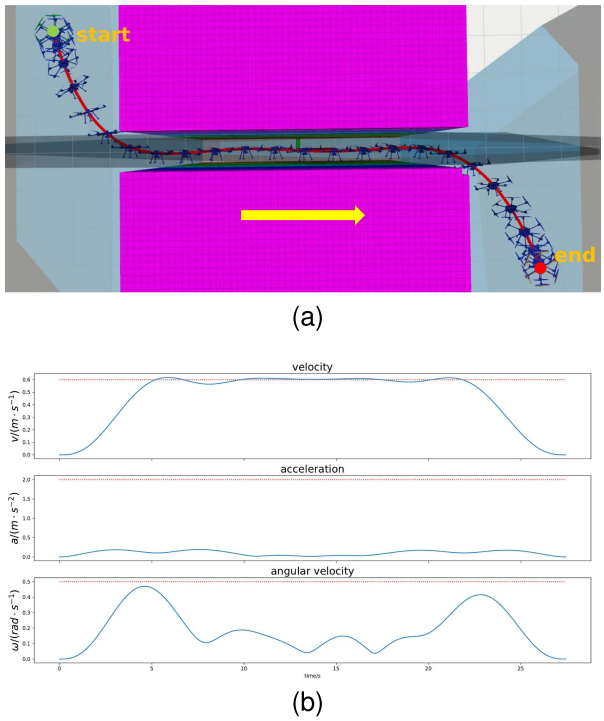


Fig. 6. A 6-D trajectory generated in Scenario A. (a) Shows the overview of the map and the trajectory (from the top view), where a series of light blue transparent convex polyhedra form the SFC, the red curve represents the geometry of the position part of the 6-D trajectory (i.e., the trajectory of CoM), and the vehicle models in dark blue represent the rotation parts of the 6-D trajectory at the corresponding sampling points. (b) Shows the kinodynamic properties of the trajectory, specifically the changes of $\|\mathbf{v}\|_2$, $\|\mathbf{a}\|_2$, and $\|\boldsymbol{\omega}\|_2$ w.r.t. time (shown by the blue curves), and the corresponding maximum limits (shown by the red dashed lines).

Considering these shortcomings, Euler angle or an axis-angle representation lacks rationality when using polynomial interpolation. In our implementation, we adopt a parameterization method based on Hamilton quaternion representation [15] and stereographic projection, which exhibits excellent numerical properties such as accelerated convergence as shown in [19], to make trajectory optimization more efficient. Utilizing the homeomorphism between the hyperplane $\mathbb{R}^3$ and the hypersphere $\mathbb{S}^3$ with one pole removed, a stereographic projection maps an arbitrary vector $\boldsymbol{\sigma}\in\mathbb{R}^3$ as a unit quaternion $\mathbf{Q}:=\begin{bmatrix}q_w & q_x & q_y & q_z\end{bmatrix}^\top=\begin{bmatrix}q_w & \mathbf{r}^\top\end{bmatrix}^\top$ representing the rotation

$$\mathbf{R}(\mathbf{Q}):=(q_w^2-\mathbf{r}^\top\mathbf{r})\mathbf{I}+2\mathbf{r}\mathbf{r}^\top+2q_w\mathbf{r}^\wedge\tag{25}$$

If the pole is chosen as $\mathbf{Q}_N=\begin{bmatrix}1 & 0 & 0 & 0\end{bmatrix}^\top$, the map is expressed as follows:

$$\mathbf{Q}(\boldsymbol{\sigma}):=\begin{bmatrix}\frac{\boldsymbol{\sigma}^\top\boldsymbol{\sigma}-1}{\boldsymbol{\sigma}^\top\boldsymbol{\sigma}+1} & \frac{2\boldsymbol{\sigma}^\top}{\boldsymbol{\sigma}^\top\boldsymbol{\sigma}+1}\end{bmatrix}^\top\in\mathbb{S}^3\backslash\{\mathbf{Q}_N\},\forall\boldsymbol{\sigma}\in\mathbb{R}^3.\tag{26}$$

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 06:31:03 UTC from IEEE Xplore. Restrictions apply.

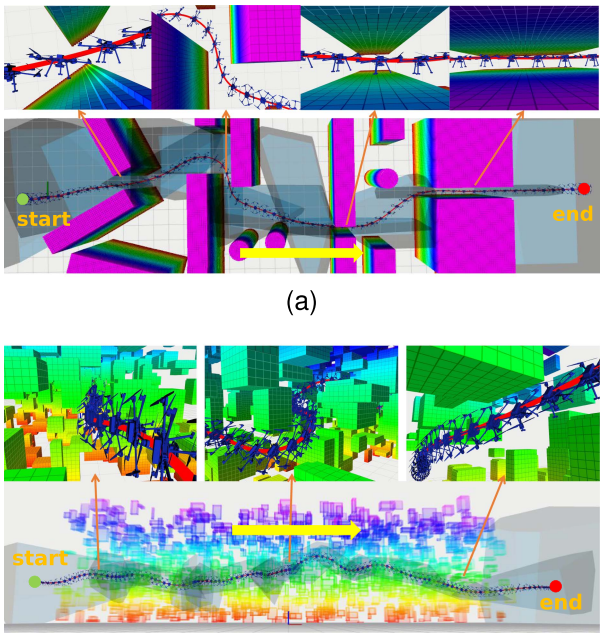


Fig. 7. Trajectory generation results. (a) Shows a trajectory generated in Scenario B (from the top view), and (b) shows that in Scenario C (from the side view). We adjusted the transparency of the point cloud visualization in the bottom image of (b) so that the generated SFC and 6-D trajectory can be displayed more clearly.

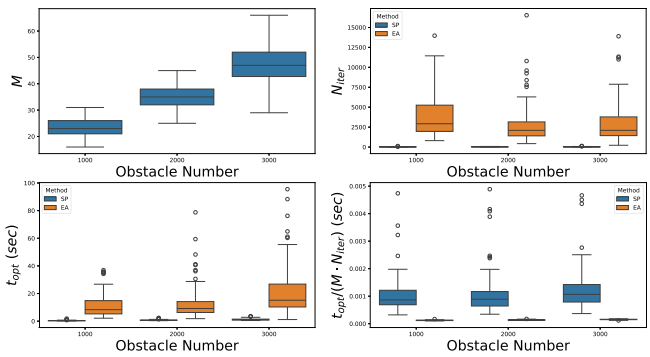


Fig. 8. The efficiency comparison between stereographic-projection-based (SP) method and Euler-angle-based (EA) method. The circles represent outliers outside the range $[Q_1 - 2 \cdot IQR, Q_3 + 2 \cdot IQR]$, where Q_1 is the lower quartile, Q_3 is the upper quartile, and $IQR = Q_3 - Q_1$.

We can see that $\mathbf{Q}(\boldsymbol{\sigma}) : \mathbb{R}^3 \to \mathbb{S}^3 \backslash \{\mathbf{Q}_N\}$ is smooth and one-to-one. Thus each rotation has at most two distinct $\boldsymbol{\sigma}$ counterparts ($\mathbf{R} = \mathbf{I}$ corresponds only to the origin of $\mathbb{R}^3$), which greatly reduces the possibility of ambiguity. The angular velocity expressed in $\mathcal{F}_W$ can be calculated as:

$$\boldsymbol{\omega} = 2\mathbf{U}\dot{\mathbf{Q}} = 2\mathbf{U}\mathbf{G}^{\top}\dot{\boldsymbol{\sigma}}, \tag{27}$$

where $\mathbf{U} := \begin{bmatrix} -\mathbf{r} & w\mathbf{I} + \mathbf{r}^{\wedge} \end{bmatrix} \in \mathbb{R}^{3\times 4}$ and $\mathbf{G} := \begin{bmatrix} \frac{\partial q_w}{\partial \boldsymbol{\sigma}} & \frac{\partial q_x}{\partial \boldsymbol{\sigma}} & \frac{\partial q_y}{\partial \boldsymbol{\sigma}} & \frac{\partial q_z}{\partial \boldsymbol{\sigma}} \end{bmatrix} \in \mathbb{R}^{3\times 4}$. Then, we can calculate the rotation-related penalty terms.

Note that inherited from [12], the stereographic projection is also used to construct the map $\mathbf{q}^p(\boldsymbol{\xi})$ due to its favorable properties.

TABLE I
OPTIMIZATION SETTINGS

κ	$v_{\max}$	$a_{\max}$	$\omega_{\max}$
16	$0.6\text{m} \cdot \text{s}^{-1}$	$2.0\text{m} \cdot \text{s}^{-2}$	$0.5\text{rad} \cdot \text{s}^{-1}$
$\mathcal{W}_v$	$\mathcal{W}_a$	$\mathcal{W}_\omega$	k_ρ
1×10^5	1×10^5	1×10^5	10

IV. SIMULATIONS AND EXPERIMENTS

In this section, we show the performance of the proposed method. First, we generate trajectories in cluttered simulation environments to verify the effectiveness and efficiency of our method. Then, real-world experiments are carried out to test the practicability of our method.

A. Implementation Details

The initial path is obtained by applying RRT to the $SE(3)$ configuration space and is used to generate SFC using RILS. The algorithm used to solve the optimization problem (24) is L-BFGS [20], with the backtracking method [21] used for line search. We implement all the trajectory generation algorithms in C++17 using a single thread. The hardware platform used for simulations is a Dell G5 laptop with an Intel Core i7-10750H CPU @ 2.60 GHz running the Ubuntu 20.04 operating system. The trajectory generation algorithms are all executed sequentially without explicit hardware acceleration.

B. Parameter Settings

To achieve whole-body obstacle avoidance, as Section III-A mentions, we approximate the vehicle's shape as a convex polyhedron that envelops the body. Here we use a cuboid (Fig. 5(a)) whose dimensions are $l_x = l_y = 1.1\,\text{m}, l_z = 0.42\,\text{m}$, according to the size of OmniHex. The coordinates of the eight vertices in $\mathcal{F}_b$ are $\tilde{\boldsymbol{v}}_l = \begin{bmatrix} \pm\frac{l_x}{2} & \pm\frac{l_y}{2} & \pm\frac{l_z}{2} \end{bmatrix}^{\top}$. Note that it is also possible to utilize a shape with closer proximity, such as a hexagon. However, the extent to which the solution space can be expanded using this type of shape rather than a cuboid is very limited, and more vertices may require additional computation. We set $s = 4$ to ensure the smoothness of control inputs according to (8). The initial value of $\mathbf{q}^{\sigma}$ of trajectory optimization is set to $\mathbf{0}$. The other optimization settings for simulations are listed in Table I.

C. Simulation Results

In this section, simulations are carried out in three virtual scenarios to test the proposed method's effectiveness and efficiency. We generate a point cloud map for each scenario to provide obstacle information for our trajectory generation framework. The three virtual scenarios are described as follows.

Scenario A: We expect the vehicle to fly through a narrow, straight, vertical passage. The passage is 0.7 m wide, which is narrower than the vehicle's width (l_x and l_y) and wider than its thickness (l_z).

Scenario B: An unstructured cluttered environment with extremely narrow regions, including the narrow passage described in Scenario A. The map is restricted to a $40\,\text{m} \times 10\,\text{m} \times 6\,\text{m}$ area.

Scenario C: We expect the vehicle to pass through a $30\,\text{m} \times 10\,\text{m} \times 8\,\text{m}$ area containing 1000 randomly distributed floating cuboid obstacles.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 06:31:03 UTC from IEEE Xplore. Restrictions apply.



Fig. 9. OmniHex executes the generated trajectory to fly through a narrow passage. The leftmost figure is the visualization of the generated trajectory, and the others are snapshots of the experiment. The red arrow indicates the chronological order of the snapshots.

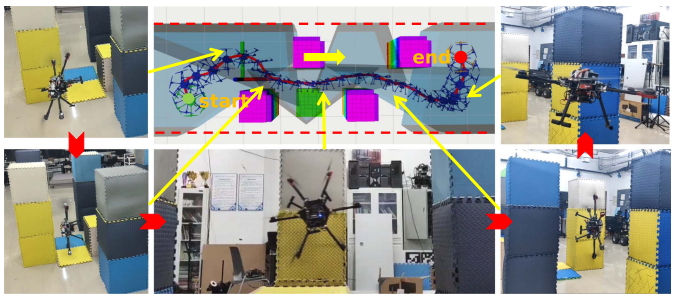


Fig. 10. OmniHex executes the generated trajectory to fly through a narrow environment. The top middle figure shows the generated trajectory where the red dashed lines represent the map's boundaries.

TABLE II
EFFICIENCY EVALUATION

scenario	$M_{\mathcal{P}}$	M	D	t_{rrt}	t_{sfc}	N_{iter}	t_{opt}
A	4	7	11.2 m	598 ms	4 ms	60	88 ms
B	12	20	34.0 m	1284 ms	64 ms	81	611 ms
C	19	24	38.1 m	55 ms	120 ms	48	422 ms

Note: t_{rrt} is the time it takes to find an initial feasible path using RRT. t_{sfc} is the time it takes to generate an SFC using RILS based on the initial path. t_{opt} is the time it takes to solve the trajectory optimization problem, and N_{iter} is the iteration number of L-BFGS.

Since there is no available 6-D trajectory generation baseline considering obstacles for OMAVs, we solely present the performance of our method.

Fig. 6(a) shows the 6-D trajectory generated in Scenario A and the corresponding kinodynamic properties under the above settings. The generated trajectory allows the vehicle to smoothly and appropriately change its attitude to adapt to the narrow space in the passage, avoiding collision while flying toward the target. Fig. 6(b) presents the constrained kinodynamic properties. Although the constraints are relaxed, they are effectively satisfied in the resulting trajectory. Moreover, the velocity norm reaches $v_{\max}$ most of the time, which shows that our method can fully exploit the performance of OMAVs. Note that due to the softening step, the resulting trajectory may slightly exceed the constraints [10]. Nonetheless, in most cases the given constraints can be well satisfied and we can handle the possible violation by carefully tuning the optimizer parameters or by adding a certain margin when approximating the shape of the robot in practice.

Fig. 7 shows the trajectories generated by the proposed method in scenarios B and C, which are much more challenging than Scenario A. The vehicle has been successfully constrained in the SFC throughout the journey and can change its attitude flexibly to avoid obstacles. The simulation results show that our method allows OMAVs to fly safely even in extremely cluttered environments. Due to space limitation, we are unableto give the corresponding kinodynamic properties of Fig. 7 in this manuscript. Please refer to our video attachment for more details.

For efficiency evaluation, we list the computation times of the results in Figs. 6 and 7 in Table II. t_{rrt} has a relatively large randomness, and in general, the narrower the feasible space, the larger t_{rrt} tends to be. t_{sfc} is positively correlated with the number of path segments and the point cloud size. Since there are alternative methods available for the initial path search and the SFC generation that can be plug and play, we do not discuss the impact of these two stages on the efficiency of our framework here. The time complexity of each iteration (where the value of the objective function and the related gradients are calculated) is $O(M)$ [12], so t_{opt} is approximately proportional to M and N_{iter}. The trajectory optimization stage shows relatively high solving efficiency, comparable to the performance on the CPU in [11], while [11] generates only 3-D position trajectories for under-actuated MAVs. Therefore, our framework have the application potential in scenarios requiring real-time performance.

In order to demonstrate the efficiency of stereographic-projection-based rotation parameterization (SP method), we also implement the Euler-Angle-based method (EA method) for additional comparison. The comparison experiment is carried out in a virtual environment that is basically the same as Scenario C, except that we set three cases for the number of obstacles: 1000, 2000, and 3000. For each case, we repeat the following steps 100 times: generate a point cloud map and an SFC, and then utilize our trajectory optimizer to perform trajectory optimization within that SFC using both SP and EA methods under the same optimization settings. Therefore, the number of trajectory pieces obtained by the two methods is equal in each trial. The results are presented in Fig. 8. From t_{opt} we can see that the SP method exhibits significantly higher efficiency than the EA method with a more concentrated time distribution and much fewer iterations for convergence. In addition, an increase in the obstacle number will increase obstacle distribution density, leading to an increasing detour of the trajectories, which is consistent with the general increase in M with obstacle number as shown in Fig. 8. On the other hand, $t_{\text{opt}}/(M \cdot N_{\text{iter}})$ is generally stable, which is consistent with the linear complexity of the trajectory optimization algorithm. In general, compared with EA method, SP method has higher convergence speed and is more suitable for real-time applications.

D. Real-World Experiments

The results presented in this section aim to verify the applicability of the proposed method can be applied to real OMAVs.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 06:31:03 UTC from IEEE Xplore. Restrictions apply.

We set up cuboid obstacles with known sizes and positions in the environment. Then, we generate collision-free 6-D trajectories in advance using the proposed method. The experimental platform is the OmniHex (Fig. 5(b)) mentioned in Section II-C. Six Dynamixel XH430-W210-T servos each provide one rotor's degree of tilt freedom. An OptiTrack motion capture system provides high-precision real-time 6-D pose feedback to the vehicle at 100 Hz via WiFi. For the trajectory optimization settings, we set the kinodynamic constraints according to the real vehicle's performance as : $v_{\max} = 0.36\,\mathrm{m}\cdot\mathrm{s}^{-1}, a_{\max} = 0.5\,\mathrm{m}\cdot\mathrm{s}^{-2}, \omega_{\max} = 1.0\ \mathrm{rad}\cdot\mathrm{s}^{-1}$.

Fig. 9 shows OmniHex executing the generated trajectory, flying through a narrow passage that is 0.7 m wide and 1.2 m long. OmniHex must tilt a large angle to fly through the narrow passage without collision. Fig. 10 shows OmniHex executing the generated trajectory, flying through an area with several cuboid obstacles. OmniHex accurately follows the reference trajectories and flies smoothly and safely from the starting points to the target points. The real-world experiment results demonstrate the practicability of our method. Please refer to our video attachment for more details.

V. Conclusion

We present a computationally efficient 6-D trajectory generation framework which can fully exploit the obstacle avoidance potential of OMAVs. A 6-D trajectory optimization problem considering safety and kinodynamic constraints is formulated. A rational quaternion-based rotation parameterization method is adopted to achieve efficient optimization and high-quality solution. Simulations and real-world experiments are carried out to verify the performance of our method. Furthermore, our method can be applied to any platform that can do free and controlled 6-D rigid body motion, including spacecraft.

The primary limitation of our method is its lack of consideration of possible dynamic changes of the environment, such as pedestrians, which limits its scope of application. In addition, utilizing the independence of each sampling point in the penalty function, the computational efficiency of our method can be further improved with parallel computation. Moreover, we hope to enable autonomous OMAV-based aerial manipulation based on the proposed method, as well as the onboard sensing system. Achieving the above improvements and goals will be our future work.

References

[1] D. Brescianini and R. D'Andrea, "Design, modeling and control of an omni-directional aerial vehicle," in *2016 IEEE Int. Conf. Robot. Automat.*, 2016, pp. 3261–3266.

[2] M. Kamel et al., "The voliro omniorientational hexacopter: An agile and maneuverable tiltable-rotor aerial vehicle," *IEEE Robot. Automat. Mag.*, vol. 25, no. 4, pp. 34–44, Dec. 2018.

[3] F. Morbidi, D. Bicego, M. Ryll, and A. Franchi, "Energy-efficient trajectory generation for a hexarotor with dual-tilting propellers," in *2018 IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 6226–6232.

[4] M. Pantic, L. Ott, C. Cadena, R. Siegwart, and J. Nieto, "Mesh manifold based Riemannian motion planning for omnidirectional micro aerial vehicles," *IEEE Robot. Automat. Lett.*, vol. 6, no. 3, pp. 4790–4797, Jul. 2021.

[5] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *2011 IEEE Int. Conf. Robot. Automat.*, 2011, pp. 2520–2525.

[6] F. Gao, Y. Lin, and S. Shen, "Gradient-based online safe trajectory generation for quadrotor flight in complex environments," in *2017 IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2017, pp. 3681–3688.

[7] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "Ego-planner: An ESDF-free gradient-based local planner for quadrotors," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 478–485, Apr. 2021.

[8] F. Gao, L. Wang, B. Zhou, X. Zhou, J. Pan, and S. Shen, "Teach-repeat-replan: A complete and robust system for aggressive flight in complex environments," *IEEE Trans. Robot.*, vol. 36, no. 5, pp. 1526–1545, Oct. 2020.

[9] F. Gao, W. Wu, W. Gao, and S. Shen, "Flying on point clouds: Online trajectory generation and autonomous navigation for quadrotors in cluttered environments," *J. Field Robot.*, vol. 36, no. 4, pp. 710–733, 2019.

[10] S. Yang, B. He, Z. Wang, C. Xu, and F. Gao, "Whole-body real-time motion planning for multicopters," in *2021 IEEE Int. Conf. Robot. Automat.*, 2021, pp. 9197–9203.

[11] Z. Han, Z. Wang, N. Pan, Y. Lin, C. Xu, and F. Gao, "Fast-racing: An open-source strong baseline for SE(3) planning in autonomous drone racing," *IEEE Robot. Automat. Lett.*, vol. 6, no. 4, pp. 8631–8638, Oct. 2021.

[12] Z. Wang, X. Zhou, C. Xu, and F. Gao, "Geometrically constrained trajectory optimization for multicopters," *IEEE Trans. Robot.*, vol. 38, no. 5, pp. 3259–3278, Oct. 2022.

[13] Y. Ren, S. Liang, F. Zhu, G. Lu, and F. Zhang, "Online whole-body motion planning for quadrotor using multi-resolution search," in *2023 IEEE Int. Conf. Robot. Automat.*, 2023, pp. 1594–1600.

[14] S. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Dept. Comput. Sci., Iowa State Univ., Ames, IA, USA, Res. Rep. 9811, 1998.

[15] J. Solà, "Quaternion kinematics for the error-state kalman filter," 2017, *arXiv:1711.02508*.

[16] M. Fliess, J. Lévine, P. Martin, and P. Rouchon, "Flatness and defect of non-linear systems: Introductory theory and examples," *Int. J. Control*, vol. 61, no. 6, pp. 1327–1361, 1995.

[17] K. Bodie, Z. Taylor, M. Kamel, and R. Siegwart, "Towards efficient full pose omnidirectionality with overactuated mavs," in *Proc. 2018 Int. Symp. Exp. Robot.*, 2020, pp. 85–95.

[18] S. Liu et al., "Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-D complex environments," *IEEE Robot. Automat. Lett.*, vol. 2, no. 3, pp. 1688–1695, Jul. 2017.

[19] G. Terzakis et al., "On quaternion based parametrization of orientation in computer vision and robotics," *J. Eng. Sci. Technol. Rev.*, vol. 6, pp. 15–24, 2013.

[20] D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," *Math. Program.*, vol. 45, no. 1–3, pp. 503–528, 1989.

[21] S. P. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge U.K.: Cambridge Univ. Press, 2004.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on October 13,2026 at 06:31:03 UTC from IEEE Xplore. Restrictions apply.